

React Interview Prep

Topic 11 — useReducer: Complete Guide

Analogy · Syntax · Full Cart Example · Action Creators · Context · Lazy Init · Immer · Is It Heavy?

- Covers All examples from conversation — nothing skipped
- Series React Interview Prep — Session 11

by Akshay Chavhan • React Interview Series

1. Like You're 5 — TV Remote Analogy



Instead of calling `setState` directly — you dispatch an action and a reducer function decides what the new state should be.

2. Why `useReducer` Exists — The Problem

Multiple related states that must update together — `useState` forces you to call multiple setters and it is easy to forget one.

■ `useState` (messy)

```
// multiple separate states – hard to sync

const [items, setItems] = useState([]);

const [total, setTotal] = useState(0);

const [discount, setDiscount] = useState(0);

const [loading, setLoading] = useState(false);

const [error, setError] = useState(null);

function addItem(item) {
  setItems(prev => [...prev, item]);
  setTotal(prev => prev + item.price);
  setDiscount(0); // easy to forget
  // must update ALL 3 every time
}
```

■ `useReducer` (clean)

```
// one state object, one dispatch
const [state, dispatch] = useReducer(cartReducer, {
  items: [], total: 0,
  discount: 0, loading: false
});

function addItem(item) {
  dispatch({
    type: 'ADD_ITEM',
    payload: item
  }); // reducer handles ALL sync
}
```

■ **Multiple related states updating together = use `useReducer`. One dispatch = all fields update correctly. Nothing forgotten.**

3. Syntax — The 4 Parts

```
const [state, dispatch] = useReducer(reducer, initialState);
// ↑↑↑↑
// current send an function starting
// state action to decides value
// reducer new state
```

4. Full Example — Shopping Cart

Complete working example with all 3 parts: initial state, reducer, and dispatch.

Part 1 — Initial State

```
const initialState = {
  items: [],
  total: 0,
  discount: 0,
  loading: false,
  error: null,
};
```

Part 2 — Reducer Function

```
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM':
      return {
        ...state, // keep existing fields
        items: [...state.items, action.payload],
        total: state.total + action.payload.price,
        discount: 0, // reset discount on add
      };
    case 'REMOVE_ITEM': {
      const item = state.items.find(i => i.id === action.payload);
      return {
        ...state,
        items: state.items.filter(i => i.id !== action.payload),
        total: state.total - item.price,
      };
    }
    case 'APPLY_DISCOUNT':
      return {
        ...state,
        discount: action.payload,
        total: state.total - (state.total * action.payload / 100),
      };
    case 'CLEAR_CART':
      return initialState; // reset everything at once ■
    default:
      return state; // always return state for unknown actions
  }
}
```

Part 3 — Component with dispatch

```
function ShoppingCart() {
  const [state, dispatch] = useReducer(cartReducer, initialState);
  return (
    <div>
      <p>Total: Rs.{state.total}</p>
      <p>Items: {state.items.length}</p>
      <button onClick={() => dispatch({
        type: 'ADD_ITEM',
```

```
payload: { id: 1, name: 'Phone', price: 999 }  
}}}>Add Phone</button>  
  
<button onClick={() => dispatch({  
  type: 'REMOVE_ITEM', payload: 1  
})}>Remove</button>  
  
<button onClick={() => dispatch({  
  type: 'APPLY_DISCOUNT', payload: 10  
})}>Apply 10% Off</button>  
  
<button onClick={() => dispatch({ type: 'CLEAR_CART' })}>  
Clear Cart</button>  
</div>  
  
</>  
</>
```

5. useState vs useReducer — When to Switch

Use useState when	Use useReducer when
1-2 independent values	3+ related values that update together
Simple toggle / string / number	Object with multiple fields
Next state doesn't depend on others	State fields depend on each other
Simple logic	Complex update logic (many cases)
No state machines needed	Finite state machine patterns

6. Action Creators — Cleaner dispatch calls

Functions that return action objects — avoids typos in type strings.

■ Wrong

```
// ■ Raw dispatch – easy to typo
dispatch({
  type: 'ADD_ITEM',
  payload: item
});

dispatch({
  type: 'REMOV_ITEM', // typo!
  payload: id
});
```

■ Correct

```
// ■ Action creators – type safe
const addItem = (item) => ({
  type: 'ADD_ITEM', payload: item
});

const removeItem = (id) => ({
  type: 'REMOVE_ITEM', payload: id
});

const clearCart = () => ({
  type: 'CLEAR_CART'
});

// Usage – clean!
dispatch(addItem(item));
dispatch(removeItem(1));
dispatch(clearCart());
```

7. useReducer + useContext — Mini Redux Pattern

Combine both for app-wide state without any external library.

```
const CartContext = createContext(null);
// 1. Provider wraps useReducer
export function CartProvider({ children }) {
  const [state, dispatch] = useReducer(cartReducer, initialState);
  const value = useMemo(
    () => ({ state, dispatch }),
    [state]
  );
  // dispatch is ALWAYS stable – never changes reference!
  return (
    <CartContext.Provider value={value}>
```

```

{children}
</CartContext.Provider>
);
}

// 2. Custom hook with safety check
export function useCart() {
  const context = useContext(CartContext);
  if (!context) throw new Error('useCart must be inside CartProvider');
  return context;
}

// 3. Any component reads state + dispatches actions
function ProductPage() {
  const { state, dispatch } = useCart();
  return (
    <div>
      <p>Cart: {state.items.length} items</p>
      <button onClick={() => dispatch(addItem(product))}>
        Add to Cart
      </button>
    </div>
  );
}

```

■ *This pattern = Mini Redux. No library needed. dispatch is ALWAYS a stable reference — never needs useCallback.*

8. Lazy Initialization

Third argument to `useReducer` — runs only ONCE on mount for expensive initial state.

■ Wrong

```
// ■ expensiveCalc runs every
render

const [state, dispatch] =
  useReducer(reducer,
    expensiveCalc());

// expensiveCalc() called on every
// render — not just first mount!
```

■ Correct

```
// ■ init function runs ONCE on
mount

function init(initialCount) {
  return { count: initialCount,
    history: [] };
}

const [state, dispatch] =
  useReducer(reducer, 0, init);
// 0 is passed to init() once only
```

9. Immer — Simplify Reducers

Immer lets you write 'mutating' code inside reducers — it handles immutability behind the scenes.

■ Wrong

```
// ■ Without Immer — manual spread

case 'ADD_ITEM':
  return {
    ...state,
    cart: {
      ...state.cart,
      items: [
        ...state.cart.items,
        action.payload
      ]
    }
  };

// gets ugly with deeply nested
state
```

■ Correct

```
// ■ With Immer — write mutating
code

import { produce } from 'immer';

const cartReducer = produce(
  (draft, action) => {
    switch (action.type) {
      case 'ADD_ITEM':
        // looks like mutation but isn't!
        draft.cart.items.push(
          action.payload
        );
        break;
    }
  }
);
```

10. Is `useReducer` Heavy? Can I Use It Anywhere?

No it is not heavy. Yes you can use it anywhere. React actually implements `useState` USING `useReducer` internally — they have identical performance.

```
// useState and useReducer = IDENTICAL performance
// useState is just a simplified useReducer under the hood!

function Counter() { // ■ small component — fine
  const [s, d] = useReducer(r, { count: 0 });
```

```

return <button onClick={() => d({type:'INC'})}>{s.count}</button>;
}

function useForm(fields) { // ■ custom hook – fine
const [state, dispatch] = useReducer(formReducer, build(fields));
return { state, dispatch };
}

function Dashboard() { // ■ multiple reducers – fine
const [filters, fDispatch] = useReducer(filterReducer, {});
const [table, tDispatch] = useReducer(tableReducer, {});
}

```

dispatch is Automatically Stable — No useCallback Needed

■ Wrong

```

// useState – need useCallback
function Parent() {
const [items, setItems] =
useState([]);

// must wrap in useCallback
const handleAdd = useCallback(()
=> {
setItems(prev => [...prev, item]);
}, []);

return <Child onAdd={handleAdd}
/>;
}

```

■ Correct

```

// useReducer – dispatch is stable
function Parent() {
const [state, dispatch] =
useReducer(reducer, initial);

// NO useCallback needed ever!
// React guarantees dispatch
// never changes reference
return (
<Child onAdd={() =>
dispatch({type:'ADD'})} />
);
}

```

■ *The choice between `useState` and `useReducer` is about CODE CLARITY — not performance. `useState` = simpler API. `useReducer` = cleaner for complex logic. Same speed.*

11. Complete Decision Guide

Situation	Best Choice
1-2 simple independent values	useState ■
Simple toggle / string / number	useState ■
3+ related fields updating together	useReducer ■
State fields depend on each other	useReducer ■
Complex transition logic	useReducer ■
Worried about performance	Either — identical ■
Passing updater to nested children	useReducer — dispatch is stable ■
Need global state without Redux	useReducer + useContext ■
Large app, DevTools needed	Redux or Zustand

12. Interview Q&A;

Q

Beginner

Q1. What is useReducer?

→ useReducer is a React hook for managing complex state. Instead of calling setState directly, you dispatch actions — plain objects describing what happened. A reducer function receives the current state and action, and returns the new state. It follows the same pattern as Redux.

Q

Beginner

Q2. When would you use useReducer over useState?

→ When state has multiple related fields that must update together, when the next state depends on multiple parts of current state, or when update logic is complex enough that multiple setStates would need to be called in sync. useState is fine for 1-2 simple independent values.

Q

Beginner

Q3. What is a reducer function?

→ A pure function that takes the current state and an action, and returns the new state. It must be pure — no side effects, no direct mutation, always returns a new object. Switch statement matches action types to decide new state. Always include a default case.

Q

Intermediate

Q4. What is dispatching an action?

→ Calling dispatch() with an action object — typically { type: 'ACTION_NAME', payload: data }. The type tells the reducer what happened. The payload carries any data needed to compute the new state. dispatch is like pressing a button on the remote — triggers the reducer.

Q

Intermediate

Q5. Is useReducer heavy or expensive?

→ No — useReducer has identical performance to useState. React actually implements useState using useReducer internally. Use it in any component, large or small. Choose based on code clarity. dispatch is always a stable reference so no useCallback needed — a bonus benefit.

Q

Intermediate

Q6. How do you combine useReducer with useContext?

→ Put useReducer inside a Context Provider and pass both state and dispatch as the context value. Any component calls useContext to read state or dispatch actions. This creates global state without Redux — the mini Redux pattern used in medium-sized apps.

Q

Expert

Q7. Why is dispatch always stable and why does it matter?

→ React guarantees the dispatch function never changes its reference between renders. Unlike setState callbacks, you never need to wrap dispatch in useCallback when passing to memoized children. This is a built-in performance benefit over useState.

Q

Expert

Q8. What is lazy initialization in useReducer?

→ Passing a third init function to useReducer: useReducer(reducer, arg, initFn). React calls initFn(arg) only on the first render to compute initial state. Use it when initial state requires expensive computation — avoids running it on every render.

Q

Expert

Q9. What is the difference between useReducer and Redux?

→ useReducer is built-in, no library needed, local to a component or shared via Context. Redux is a full library with middleware, DevTools, time-travel debugging, and strict conventions. useReducer + Context works for small-medium apps. Redux makes sense for very large teams needing DevTools or strict audit trails.